

Assignment 10

Dockerize & Deploy an Express.js App
using Docker Compose with Nginx

Submitted by: **Mahmudur Rahman**

Date: 24 June 2026

Source repository:

<https://github.com/roy35-909/Module-3-deployment>

DockerHub image:

https://hub.docker.com/r/mrkolince/mahmud_assignment10

This document is both the assignment submission and a step-by-step guide — follow the steps in order to reproduce the entire setup from scratch.

Objective

Containerize a simple Express.js server, publish its image to DockerHub, and run it alongside an Nginx container using Docker Compose. The Express app must start **after** Nginx and be reachable in the browser on **port 8080**.

What you need before starting (prerequisites)

- **Docker Desktop** installed and running on Windows (includes Docker Engine + Docker Compose v2).
- A **DockerHub account**. In this guide the username is **mrkolinec** — replace it with your own everywhere it appears.
- **Git** installed (to clone the source repository).
- Any terminal — this guide uses **Windows PowerShell**.

How the two services fit together

Docker Compose builds the Express app from our **Dockerfile** and pulls a stock **nginx:latest** image. The app container maps host port **8080** to container port **5000** (where Express listens). The **depends_on** setting makes Compose start Nginx first, then the Express app.

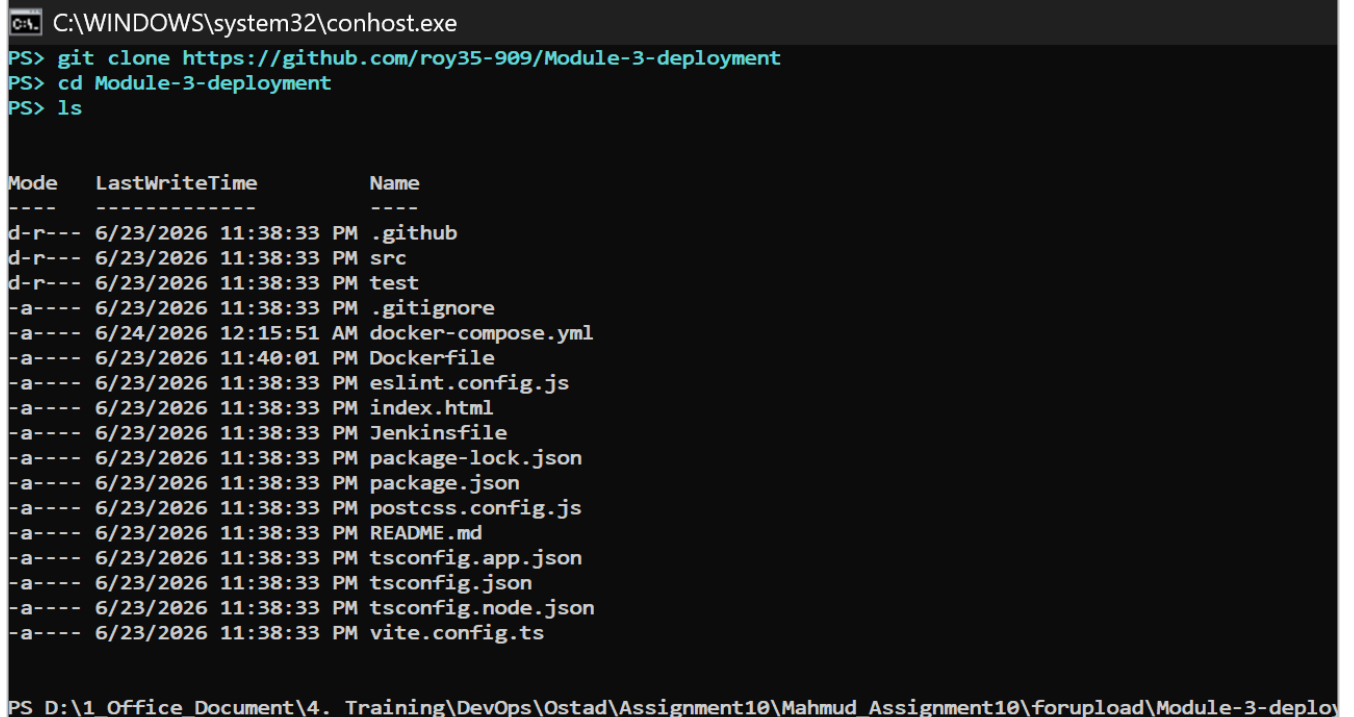
Step 1 — Clone the repository and inspect the project

Clone the provided Express.js project and move into the folder. Listing the contents confirms the files we will work with — most importantly we will add a **Dockerfile** and a **docker-compose.yml** here.

```
git clone https://github.com/roy35-909/Module-3-deployment
```

```
cd Module-3-deployment
```

```
ls
```



C:\WINDOWS\system32\conhost.exe

```
PS> git clone https://github.com/roy35-909/Module-3-deployment
PS> cd Module-3-deployment
PS> ls
```

Mode	LastWriteTime	Name
d-r---	6/23/2026 11:38:33 PM	.github
d-r---	6/23/2026 11:38:33 PM	src
d-r---	6/23/2026 11:38:33 PM	test
-a----	6/23/2026 11:38:33 PM	.gitignore
-a----	6/24/2026 12:15:51 AM	docker-compose.yml
-a----	6/23/2026 11:40:01 PM	Dockerfile
-a----	6/23/2026 11:38:33 PM	eslint.config.js
-a----	6/23/2026 11:38:33 PM	index.html
-a----	6/23/2026 11:38:33 PM	Jenkinsfile
-a----	6/23/2026 11:38:33 PM	package-lock.json
-a----	6/23/2026 11:38:33 PM	package.json
-a----	6/23/2026 11:38:33 PM	postcss.config.js
-a----	6/23/2026 11:38:33 PM	README.md
-a----	6/23/2026 11:38:33 PM	tsconfig.app.json
-a----	6/23/2026 11:38:33 PM	tsconfig.json
-a----	6/23/2026 11:38:33 PM	tsconfig.node.json
-a----	6/23/2026 11:38:33 PM	vite.config.ts

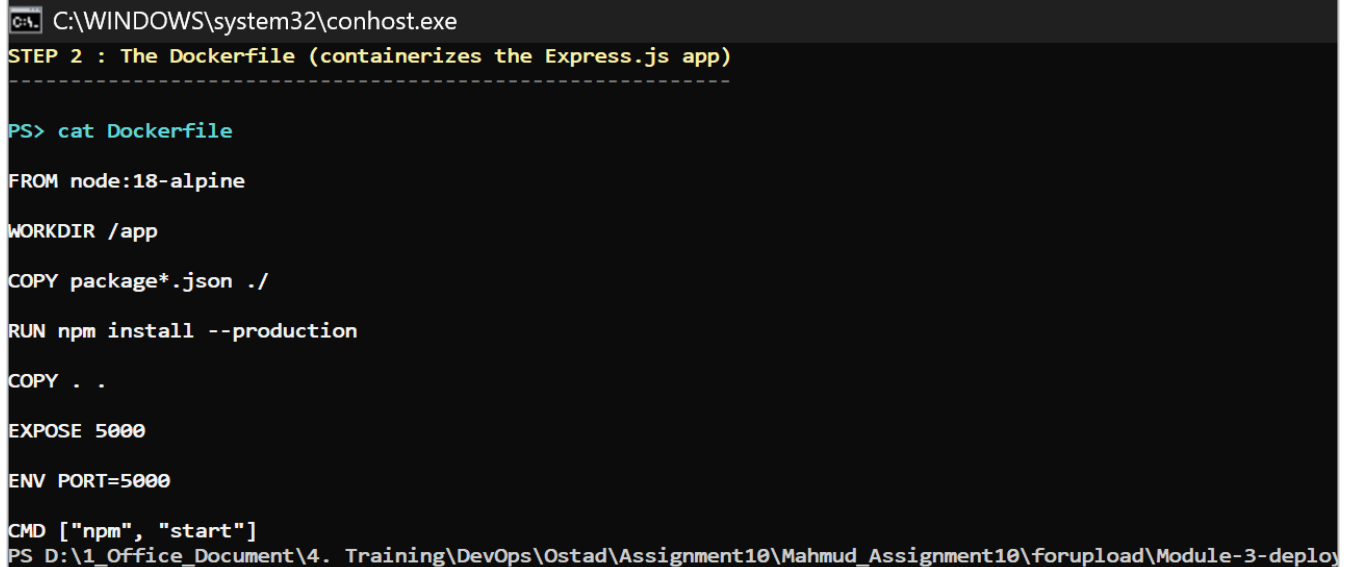
PS D:_Office_Document\4. Training\DevOps\Ostad\Assignment10\Mahmud_Assignment10\forupload\Module-3-deploy

The cloned project files.

Step 2 — The Dockerfile (containerize the Express app)

Create a file named **Dockerfile** in the project root with the content shown below. It starts from a lightweight Node 18 image, installs only production dependencies, copies the source, exposes port 5000 and starts the app with `npm start`.

```
cat Dockerfile
```



The screenshot shows a terminal window with the title bar "C:\WINDOWS\system32\conhost.exe". The terminal content is as follows:

```
STEP 2 : The Dockerfile (containerizes the Express.js app)
-----

PS> cat Dockerfile

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install --production

COPY . .

EXPOSE 5000

ENV PORT=5000

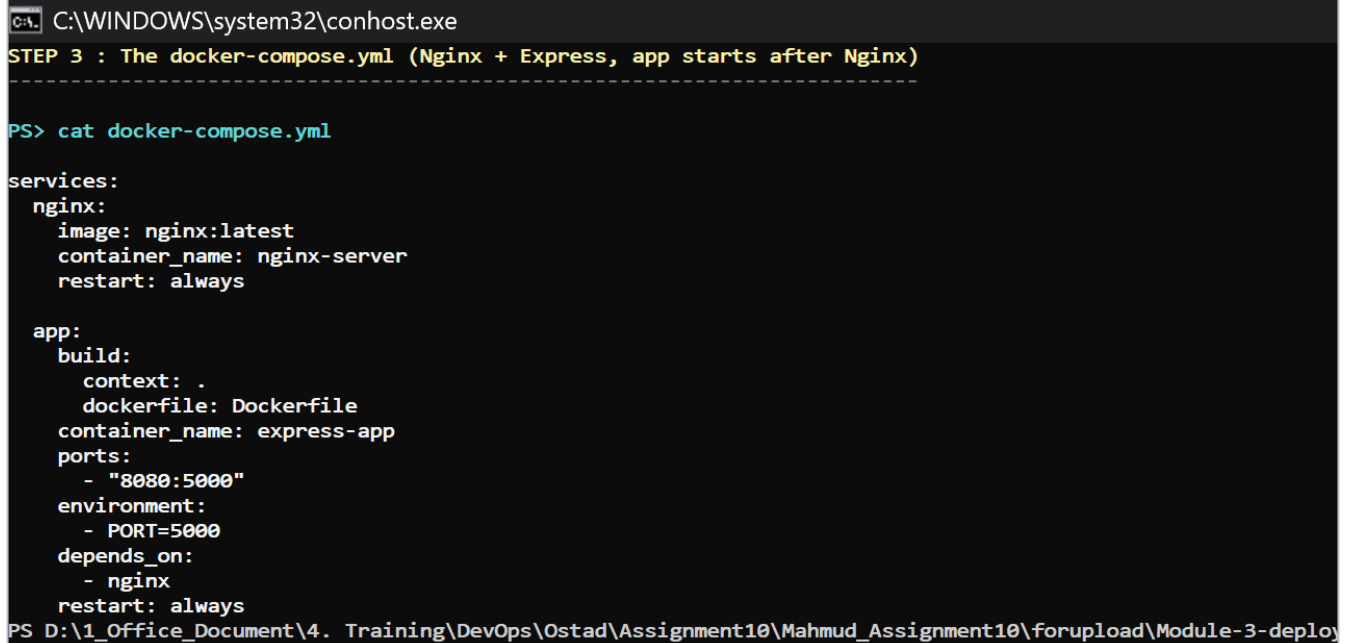
CMD ["npm", "start"]
PS D:\1_Office_Document\4. Training\DevOps\Ostad\Assignment10\Mahmud_Assignment10\forupload\Module-3-deploy
```

The Dockerfile used to build the Express.js image.

Step 3 — The docker-compose.yml

Create **docker-compose.yml** in the project root. It defines two services: **nginx** (a plain nginx:latest image) and **app** (built from our Dockerfile). The app maps 8080:5000 and uses `depends_on: nginx` so the Express app starts after Nginx.

```
cat docker-compose.yml
```



A terminal window titled 'C:\WINDOWS\system32\conhost.exe' showing the command 'cat docker-compose.yml' and its output. The output is a YAML configuration for a Docker Compose file. It defines two services: 'nginx' and 'app'. The 'nginx' service uses the 'nginx:latest' image, has a container name of 'nginx-server', and restarts 'always'. The 'app' service is built from the current directory ('context: .') using the 'Dockerfile', has a container name of 'express-app', maps port 8080 to 5000, sets the environment variable 'PORT=5000', depends on the 'nginx' service, and restarts 'always'. The terminal prompt is 'PS>' and the current directory is 'D:\1_Office_Document\4. Training\DevOps\Ostad\Assignment10\Mahmud_Assignment10\forupload\Module-3-deploy'.

```
C:\WINDOWS\system32\conhost.exe
STEP 3 : The docker-compose.yml (Nginx + Express, app starts after Nginx)
-----
PS> cat docker-compose.yml

services:
  nginx:
    image: nginx:latest
    container_name: nginx-server
    restart: always

  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: express-app
    ports:
      - "8080:5000"
    environment:
      - PORT=5000
    depends_on:
      - nginx
    restart: always
PS D:\1_Office_Document\4. Training\DevOps\Ostad\Assignment10\Mahmud_Assignment10\forupload\Module-3-deploy
```

The docker-compose.yml defining Nginx + Express.

Step 4 — Build the Docker image

Build the image and tag it with your DockerHub namespace so it is ready to push. After the build, docker images confirms the image exists locally (~192 MB).

```
docker build -t mrkolince/mahmud_assignment10:latest .
```

```
docker images mrkolince/mahmud_assignment10
```

```
C:\WINDOWS\system32\conhost.exe
STEP 4 : Build the Docker image
-----

PS> docker build -t mrkolince/mahmud_assignment10:latest .

[+] Building 2.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 186B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9
=> [internal] load build context
=> => transferring context: 3.33kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install --production
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:0f03c0f00b704de27517e8c26ed95d247fae2d6c64393cb5c9003cfd9ddf7f19
=> => exporting config sha256:1ede48c4c8e86116ed2eb4899bf6353d840c4add73494120960716730e99051c
=> => exporting attestation manifest sha256:63095c2abb702086c60946c665d5fb6f36b16638d3148e77401469b63acf7d
=> => exporting manifest list sha256:62ee5a9b327951078e61b5e9ae1f55316d02c93d2e5552ce0aa57c2c1eb51730
=> => naming to docker.io/mrkolince/mahmud_assignment10:latest
=> => unpacking to docker.io/mrkolince/mahmud_assignment10:latest

PS> docker images mrkolince/mahmud_assignment10
```

IMAGE	TD	SIZE	IMAGE	CONTENT	SIZE	EXTRA
mrkolince/mahmud_assignment10:latest		192 MB				

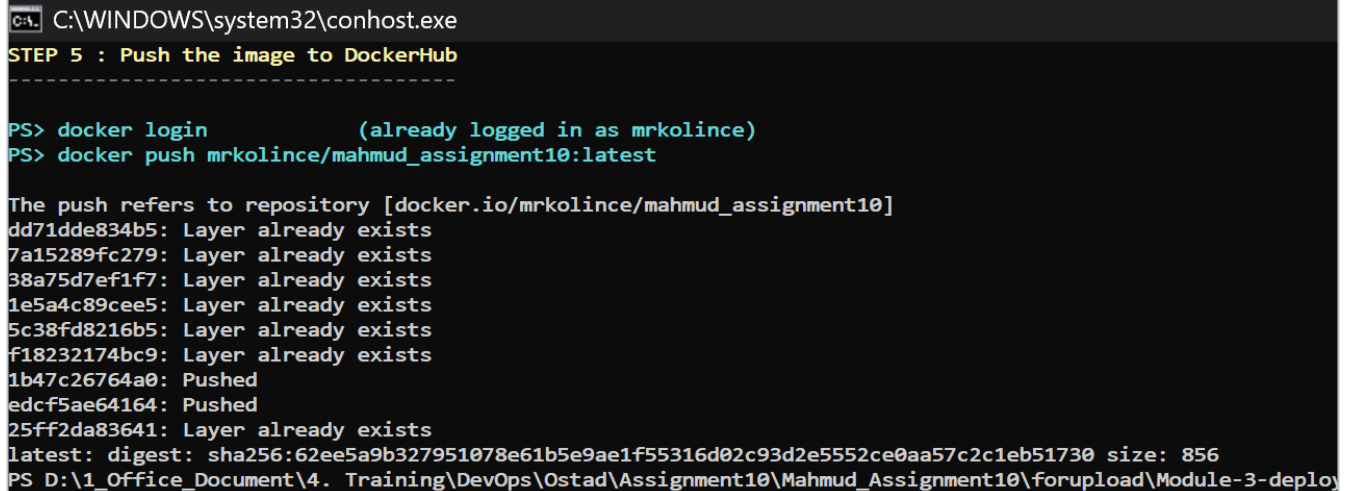
A successful build; the image is listed at the bottom.

Step 5 — Push the image to DockerHub

Log in once with `docker login` (enter your DockerHub username and password when prompted), then push the tagged image. The final `digest: sha256:...` line confirms the upload succeeded.

```
docker login
```

```
docker push mrkolince/mahmud_assignment10:latest
```



The screenshot shows a Windows command prompt window with the title bar "C:\WINDOWS\system32\conhost.exe". The prompt displays the following text:

```
STEP 5 : Push the image to DockerHub
-----

PS> docker login           (already logged in as mrkolince)
PS> docker push mrkolince/mahmud_assignment10:latest

The push refers to repository [docker.io/mrkolince/mahmud_assignment10]
dd71dde834b5: Layer already exists
7a15289fc279: Layer already exists
38a75d7ef1f7: Layer already exists
1e5a4c89cee5: Layer already exists
5c38fd8216b5: Layer already exists
f18232174bc9: Layer already exists
1b47c26764a0: Pushed
edcf5ae64164: Pushed
25ff2da83641: Layer already exists
latest: digest: sha256:62ee5a9b327951078e61b5e9ae1f55316d02c93d2e5552ce0aa57c2c1eb51730 size: 856
PS D:\1_Office_Document\4. Training\DevOps\Ostad\Assignment10\Mahmud_Assignment10\forupload\Module-3-deploy
```

The image layers are pushed and the digest is returned.

Step 6 — Start everything with Docker Compose

From the project folder, bring up the whole stack in the background. Compose pulls `nginx:latest`, builds the app image, creates a network, and starts both containers — Nginx first, then the Express app.

```
docker compose up -d
```

```
C:\WINDOWS\system32\conhost.exe

[+] up 10/10
✔ Image nginx:latest Pulled
#1 [internal] load local bake definitions
#1 reading from stdin 697B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 186B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/node:18-alpine
#3 DONE 1.0s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 3.00kB 0.0s done
#5 DONE 0.0s

#6 [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9b4b1e1e1e1e1e
#6 resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9b4b1e1e1e1e1e
#6 DONE 0.0s

#7 [4/5] RUN npm install --production
#7 CACHED

#8 [2/5] WORKDIR /app
#8 CACHED
```

Nginx pulled, app built, both containers started.

Step 7 — Verify the running containers

Confirm both containers are **Up**. Note the app's port mapping `0.0.0.0:8080->5000/tcp` and that `nginx-server` exposes `80/tcp` internally.

```
docker compose ps
```

```
docker ps
```

```
C:\WINDOWS\system32\conhost.exe
STEP 7 : Verify the running containers
-----

PS> docker compose ps

NAME                IMAGE                COMMAND              SERVICE    CREATED        STATUS
express-app         module-3-deployment-app  "docker-entrypoint.s..."  app        30 seconds ago  Up 29 seconds
nginx-server         nginx:latest           "/docker-entrypoint..."  nginx      30 seconds ago  Up 30 seconds

PS> docker ps

NAMES                IMAGE                STATUS              PORTS
express-app         module-3-deployment-app  Up 30 seconds      0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp
nginx-server         nginx:latest           Up 30 seconds      80/tcp

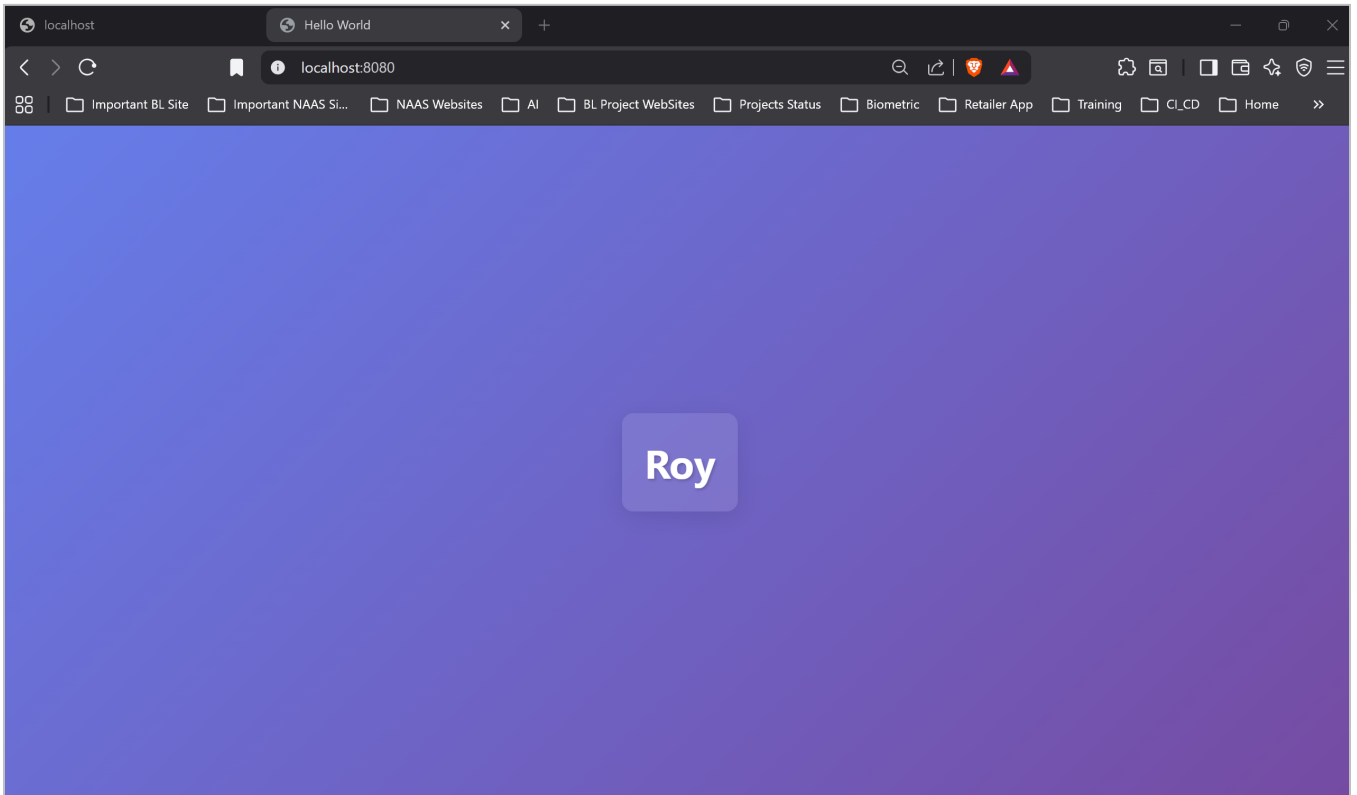
Express app -> http://localhost:8080 (host 8080 -> container 5000)
Nginx        -> nginx:latest running, port 80/tcp (internal)
PS D:\1_Office_Document\4. Training\DevOps\Ostad\Assignment10\Mahmud_Assignment10\forupload\Module-3-deploy
```

Both containers running; app exposed on 8080.

Step 8 — Verify it works in the browser

Open a browser and visit **http://localhost:8080**. The Express app responds — proving the containerized application is reachable on the required port 8080.

```
start http://localhost:8080
```

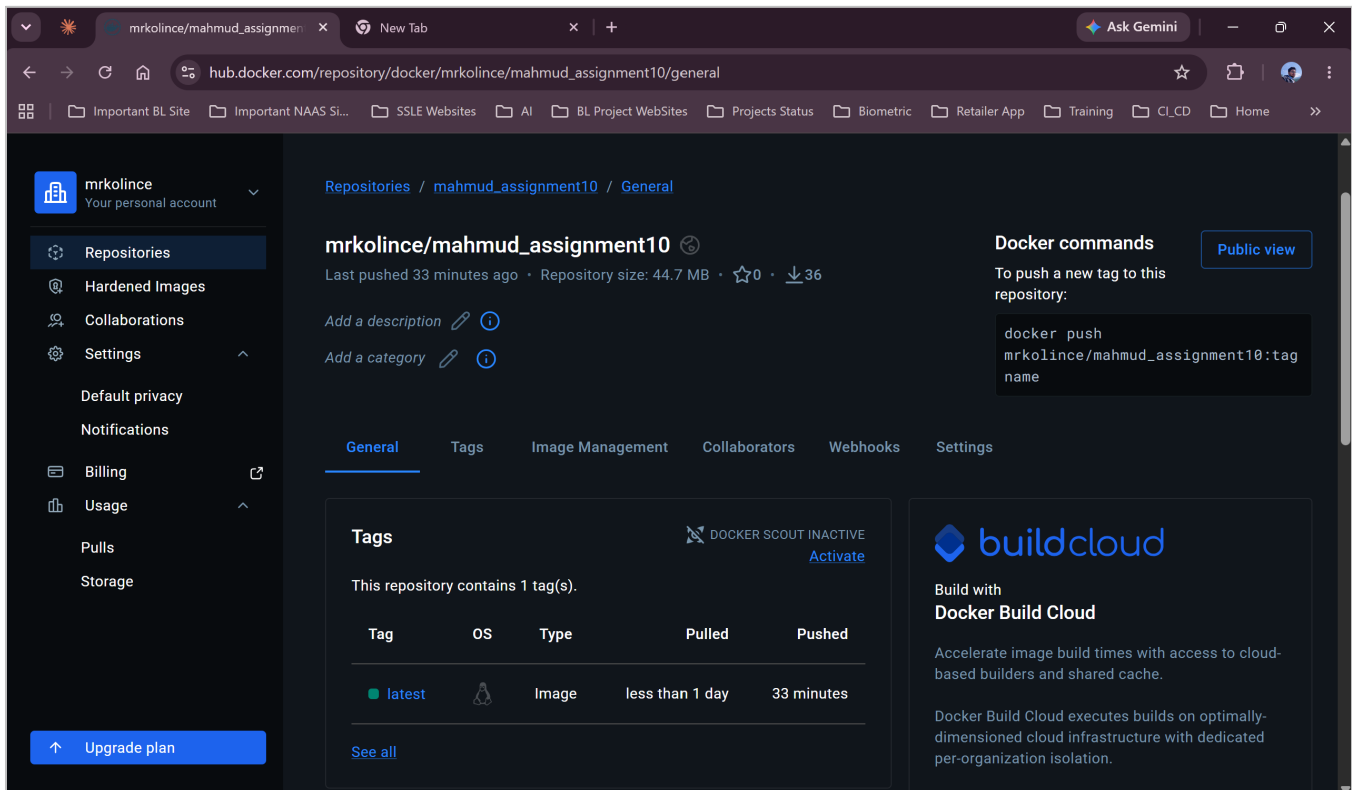


The app served from the container at localhost:8080.

Step 9 — Confirm the image on DockerHub

Finally, open your DockerHub repository to confirm the image was published. The **latest** tag is present and anyone can now pull it with the command shown on the page.

```
docker pull mrkolince/mahmud_assignment10:latest
```



The screenshot shows the DockerHub repository page for `mrkolince/mahmud_assignment10`. The page is divided into several sections:

- Header:** Shows the repository name `mrkolince/mahmud_assignment10` and the general tab selected.
- Repository Info:** Displays "Last pushed 33 minutes ago", "Repository size: 44.7 MB", and "36" downloads.
- Docker commands:** Provides the command to push a new tag: `docker push mrkolince/mahmud_assignment10:tag name`.
- Tags:** A table showing the tags in the repository.
- Build Cloud:** A section promoting Docker Build Cloud.

Tag	OS	Type	Pulled	Pushed
latest	linux	Image	less than 1 day	33 minutes

The published image on DockerHub (latest tag).

Step 10 — Docker Desktop view (optional)

Docker Desktop → **Containers** shows the running module-3-deployment Compose stack with a green 'running' indicator — a visual confirmation of the same state as the terminal in Step 7.

The screenshot shows the Docker Desktop interface with the 'Containers' tab selected. The left sidebar contains navigation options: Gordon, Containers, Images, Volumes, Kubernetes, Builds, Models, MCP Toolkit (BETA), Docker Hub, Logs, and Extensions. The main area displays container statistics and a table of running containers.

Container CPU usage: 0.00% / 1800% (18 CPUs available)
Container memory usage: 48.89MB / 7.31GB

Search: [] Only show running containers []

		Name	Container ID	Image	Port(s)	CPU (%)	Memc	Actions
☐	▼	module-3-deplo	-	-	-	0%	48.89	
☐		nginx-server	9495b5ca5984	nginx:lates		0%	15.99	
☐		express-app	a93ea9dc4aa7	module-3-c	8080:5000	0%	32.9M	

Upgrade plan []

Preparing to install update. [X]

Engine running | RAM 1.20 GB CPU 0.06% Disk: 1.93 GB used (limit 1006.85 GB) > Update version

The Compose stack running, seen in Docker Desktop.

Submission summary

Item	Link / Value
DockerHub image	https://hub.docker.com/r/mrkolince/mahmud_assignment10
Pull command	<code>docker pull mrkolince/mahmud_assignment10:latest</code>
Git repository	https://github.com/roy35-909/Module-3-deployment
Files added to repo	Dockerfile, docker-compose.yml
App URL (local)	<code>http://localhost:8080</code>
Exposed port	8080 (host) -> 5000 (container)

All five assignment objectives are satisfied: the Express app is dockerized and pushed to DockerHub; docker-compose.yml runs an Nginx image and builds/runs the Express app with depends_on; the app is exposed on port 8080; everything is run via Docker Compose; and it is verified working in the browser.